

```

Vuelo ES seq < IR >
millasGanadas ES seq < struct < millas: Millas, nombrePromo: string, factorPromo: IR > >
millasCanjeadas ES seq < IR >
millasRecibidas ES seq < struct < transferidos: Usuario, millas: Millas > >
millasTransferidas ES seq < struct < receptor: Usuario, millas: Millas > >

enum Categoria ES { Bronce, Plata, Oro, Platino }

Transacciones ES struct < distanciaVolada, millasGanadas, millasCanjeadas, millasRecibidas, millasTransferidas >.
Usuarios ES dicc < usuario: Usuario, transacciones: Transacciones >
Características ES struct < factor: IR, tope: IR >
PromoConTope ES struct < nombre: string, características: Características >
Promo ES struct < nombre: string, factor: IR >
usuario ES  $\mathbb{Z}$ 
millas ES  $\mathbb{R}$ 
Promos ES seq < Promo >
PromosConTope ES seq < PromoConTope >

```

```

TAD AirMiles {
  obs usuarios: Usuarios
  obs promos: Promos
  obs promosConTope: PromosConTope
}

```

```

proc clearSistema ( ) : AirMiles
  requiere { True }
  asegura { res.usuarios = { }  $\wedge$  res.promos = seq << nombre: 'NoPromo', factor: 1 >>  $\wedge$  res.promosConTope = < > }

```

```

proc registrarUsuario ( inout aerolínea: AirMiles, in usuario: Usuario ) {
  requiere { usuario  $\notin$  aerolínea.usuarios  $\wedge$  aerolínea = aerolínea0 }
  asegura { |aerolínea.usuarios| = |aerolínea0.usuarios| + 1  $\wedge$ 

```

```

  aerolínea.usuarios = SetKey ( aerolínea.usuarios0, usuario, < vuelos: < >, millasGanadas: < >, millasCanjeadas: < >,
    millasRecibidas: < >, millasTransferidas: < > > )  $\wedge$ 
  aerolínea.promos = aerolínea0.promos  $\wedge$  aerolínea.promosConTope = aerolínea0.promosConTope

```

```

proc registrarUsuarioConReferidor (inout aerolinea: AirMiles, in usuario: Usuario, in referidor: Usuario) {
  requiere { aerolinea = aerolinea0  $\wedge$  usuario  $\notin$  aerolinea.usuarios  $\wedge$  referidor  $\in$  aerolinea.usuarios }

```

```

  asegura { |aerolinea.usuarios| = |aerolinea.usuarios| + 1  $\wedge$ 

```

```

  aerolinea.usuarios = setkey(aerolinea0.usuario, usuario,
    < vuelos: < >,

```

```

    millasGanadas: < 500, nombrePromo: aerolinea0.usuarios[referidor].millasGanadas.nombrePromo, factorPromo: aerolinea0.usuarios[referidor].millasGanadas.factorPromo
    millasCanjeadas: < >, millasRecibidas: < >, millasTransferidas: < > >

```

```

 $\wedge$  aerolinea.usuarios = setkey(agregarUsuario(aerolineas0.usuarios, usuario), referidor, < vuelos: aerolinea0.usuarios[referidor].vuelos .

```

```

 $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoContope = aerolinea0.promoContope

```

```

aux agregarUsuarioReferido (usuarios: Usuarios, usuario: Usuario) : Usuarios

```

```

  setkey (usuarios, usuario, < vuelos: < >, millasGanadas: < millas: 500, nombrePromo: 'NoPromo', factorPromo: 1 >, millasCanjeadas: < >,
    millasRecibidas: < >, millasTransferidas >

```

```

proc acumularMillas (inout aerolínea: AirMiles, in usuario: Usuario, in distanciaVolada: Vuelo, in promo: Promo)
  requiere { aerolínea = aerolínea0 ∧ usuario ∈ aerolínea.usuarios ∧ promo ∈ aerolínea.promos ∧ distanciaVolada > 0 }

  asegura { aerolínea.usuarios = SetKey (aerolínea0.usuarios, usuario, < concat (aerolínea0.usuarios[usuario].vuelos, < distanciaVolada>,
    concat (aerolínea0.usuarios[usuario].millas Ganadas, < agregarMillas Ganadas (aerolínea0.usuarios[usuario], promo, distanciaVolada)>,
    millas Canjeadas: aerolínea0.usuarios[usuario].millas Canjeadas,
    aerolínea0.usuarios[usuario].millas Recibidas,
    aerolínea0.usuarios[usuario].millas Transferidas > .

  ∧ aerolínea.promo = aerolínea0.promo ∧ aerolínea.promoContope = aerolínea0.promoContope }

```

```

aux agregarMillasGanadas (usuario: Usuario, distanciaVolada: Vuelo, transacciones: Transacciones, promo: Promo): Millas Ganadas
  < Millas: CalcularMillasAcumuladas (usuario, distanciaVolada, transacciones, promo), NombrePromo: promo.nombre, factorPromo: promo.factor >

```

```

aux calcularMillasAcumuladas (usuario: Usuario, distanciaVolada: Vuelo, transacciones: Transacciones, promo: Promo): IR
  distanciaVolada * factorCategoría(transacciones) * promo.factor.

```

```

aux factorCategoría (transacciones: Transacciones)

```

```

IfThenElse (esBronce (millasTotales(transacciones), 1.0, IfThenElse (esPlata (millasTotales(transacciones), 1.5,
  IfThenElse (esOro (millasTotales(transacciones), 2.0, 0))))

```

```

pred esBronce (millasTotales: IR)
  millasTotales < 10.000

```

```

pred esPlata (millasTotales: IR)
  10.000 ≤ millasTotales < 50.000

```

```

pred esOro (millasTotales: IR)
  50.000 ≤ millasTotales < 1.000.000

```

```

proc canjearMillas (inout aerolinea : AirMiles, usuario : Usuario, millasACanjar : Millas)
  requiere { aerolinea = aerolinea0  $\wedge$  usuario  $\in$  aerolinea.usuarios  $\wedge$  millasACanjar  $\leq$  millasDisponibles (aerolinea0.usuarios[usuario]) }
  asegura { aerolinea.usuarios = SetKey (aerolinea0.usuarios, usuario, < Vuelos: aerolinea0.usuarios[usuario].Vuelos,
    MillasGanadas: // // . MillasGanadas,
    MillasCanjeadas: Concat (aerolinea0.usuarios[usuario].MillasCanjeadas, <millasACanjar>),
    MillasRecibidas: // // . MillasRecibidas,
    MillasTransferidas: // // . MillasTransferidas > )
     $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoContope = aerolinea0.promoContope }

```

```

proc transferirMillas (inout aerolinea : AirMiles, in usuarioTransferidor : Usuario, in usuarioReceptor : Usuario, in millasATransferir : Millas)
  requiere { aerolinea = aerolinea0  $\wedge$  usuarioTransferidor  $\in$  aerolinea.usuarios  $\wedge$  usuarioReceptor  $\in$  claves (aerolinea.usuarios)  $\wedge$ 
    millasATransferir  $\leq$  millasDisponibles (aerolinea0.usuarios[usuarioTransferidor]) }
  asegura { aerolinea.usuarios = SetKey (agregarTransferenciaRecibida (aerolinea0.usuarios, usuarioTransferidor, usuarioReceptor, millasATransferir),
    usuarioTransferidor, < Vuelos: usuarios[usuarioReceptor].Vuelos,
    millasGanadas: // // . MillasGanadas
    millasCanjeadas: // // . MillasCanjeadas
    millasRecibidas: // // . MillasRecibidas
    millasTransferidas: Concat (usuarios[usuarioTransferidor].MillasTransferidas, <receptor: usuarioReceptor,
    millasTransferidas: millasATransferir > )
     $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoContope = aerolinea0.promoContope }

```

```

aux agregarTransferenciaRecibida (usuarios: Usuarios, usuarioTransferidor : Usuario, usuarioReceptor : Usuario, millasATransferir : Millas) =
  setKey (usuarios, usuarioReceptor, < Vuelos: usuarios[usuarioReceptor].Vuelos,
    millasGanadas: // // . MillasGanadas
    millasCanjeadas: // // . MillasCanjeadas
    millasRecibidas: Concat (usuarios[usuarioReceptor].MillasRecibidas, <transferidor: usuarioTransferidor,
    millasTransferidas: // // . MillasTransferidas millasRecibidas: millasATransferir > )

```

proc consultarSaldo (in aerolínea: AirMiles, in usuario: Usuario) : Millas
 requiere : { usuario $\in$ aerolínea.usuarios
 asegura : { res = millasDisponibles (aerolínea.usuarios[usuario])

proc consultarCategoria (in aerolínea: AirMiles, usuario: Usuario)
 requiere : { usuario $\in$ aerolínea.usuarios
 asegura : { res = IfThenElse (esBronce (millasTotales (transacciones), Bronce ,
 IfThenElse (esPlata (millasTotales (transacciones), Plata ,
 IfThenElse (esOro (millasTotales (transacciones) , Oro , Platino))) }

proc rankingMillas (in aerolínea: AirMiles, in cantidad: $\mathbb{Z}$) : Seq < Usuario >
 requiere : { cantidad > 0 }
 asegura : { |res| = cantidad $\wedge$ |res| $\leq$ |aerolínea.usuarios|

($\forall i: \mathbb{Z}$) ($0 \leq i < |aerolínea.usuarios| - 1$) (millasDisponibles (aero

aux (usuarios: Usuarios) : Seq < millasDisponibles Usuario >
 $\forall$ usuario [usuario]

$\left[\begin{array}{c} \text{millas} \\ \text{struct} \end{array} \right]$

MillasDisponiblesUsuario ES struct < usuario: Usuario, millas: Millas >

$$\text{aux SumarTransacciones (transacción: seq < IR >) : IR} \equiv$$

$$\sum_{i=0}^{\text{transacciones}-1} \text{transacción}[i]$$

$$\text{aux millasTotales (transacciones: Transacciones) : IR} \equiv$$

$$\text{SumarTransacciones (transacciones.MillasGanadas.Millas)} + \text{SumarTransacciones (transacciones.MillasRecibidas)}$$

$$\text{aux millasDisponibles (transacciones: Transacciones) : IR} \equiv$$

$$\text{millasTotales (transacciones)} - \text{SumarTransacciones (transacciones.MillasCanjeadas)} + \text{SumarTransacciones (transacciones.MillasRecibidas)}$$